

Assignment: SDD, BDD, and TDD in AI-Assisted Software Development

Student Information

- Name: 諾柏洛
- Student ID: 1113526
- Course: CS351
- Date: 2026/05/31
- Due Date: 2026/05/31

23:59:59

1. Introduction

Software development has changed significantly with the rise of AI coding assistants. Developers can now describe a problem and receive a working draft of code within seconds. But this speed comes with a hidden risk: AI tools generate output based on pattern matching, not genuine understanding. If a developer's instructions are vague, the AI fills in the gaps with its own assumptions — and those assumptions are often wrong.

Consider asking an AI to "build a student grade calculator." The AI might produce something that runs without errors but uses the wrong formula, ignores rounding, or crashes when a score is missing. The developer has no reliable way to catch these mistakes unless there is a pre-written standard to compare against.

This is where SDD, BDD, and TDD become essential. Specification-Driven Development (SDD) requires developers to describe every rule and constraint before writing a single line of code. Behavior-Driven Development (BDD) translates those rules into readable, scenario-based descriptions of how the system should act. Test-Driven Development (TDD) turns those scenarios into concrete, checkable test cases.

When used together, these three approaches act as a quality filter for AI-generated code. The developer defines what correct looks like first, then uses

that definition to evaluate whatever the AI produces. This shifts the developer's role from "person who asks AI for code" to "person who sets the standard and judges the result."

2. Definition of SDD

Specification-Driven Development is a practice where developers write a complete description of the system they intend to build before any implementation work begins. This description covers not just what the system does, but also what data it accepts, what it produces, and what rules it must follow under all conditions.

The purpose of a specification is to eliminate ambiguity. When a developer sits down to write a specification, they are forced to think through every detail: What happens if a user enters a negative number? What is the expected output format? What counts as a correct answer? These questions must be answered in writing before anything is built.

A well-written specification typically covers these areas:

- **Purpose:** Why does this system exist and what problem does it solve?
- **Functional scope:** What actions must the system be capable of performing?
- **Input definition:** What data does the system receive, and what are the valid ranges?
- **Output definition:** What does the system return or display, and in what format?
- **Business rules:** What formulas, logic, or constraints govern the system's behavior?
- **Acceptance criteria:** What specific conditions must be true for the system to be considered working correctly?

In an AI-assisted workflow, the specification serves as the prompt. Instead of typing a casual description, the developer can paste the specification directly into an AI tool and receive output that is far more likely to be correct — because the specification removes the guesswork.

3. SDD: Student Grade Calculator

3.1 Goal

The Student Grade Calculator exists to give students and instructors a consistent, transparent way to determine a final academic grade. Given four individual component scores, the system computes a single weighted result and maps it to a letter grade. The calculation must be reproducible — the same inputs must always produce the same output, with no ambiguity about rounding or grade boundaries.

3.2 Functional Requirements

- The system must accept exactly four numeric scores as input: one for assignments, one for the midterm exam, one for the final exam, and one for the project component.
- Each score must be validated to confirm it falls within the range of 0 to 100 inclusive before any calculation is performed.
- The system must compute a weighted average using fixed weights: assignments contribute 30%, midterm 20%, final exam 30%, and project 20%.
- The computed average must be rounded to one decimal place. This rounded value is the official final score.
- The system must determine a letter grade by comparing the rounded final score against a fixed grading scale.
- The system must present both the final score and the letter grade as its output.
- If any input score is outside the valid range, the system must halt and return an error message identifying which input was invalid.

3.3 Input

Field	Acceptable Range	Notes
Assignment score	0.0 to 100.0	Decimal values are permitted
Midterm exam score	0.0 to 100.0	Decimal values are permitted
Final exam score	0.0 to 100.0	Decimal values are permitted

Field	Acceptable Range	Notes
Project score	0.0 to 100.0	Decimal values are permitted

3.4 Output

Field	Format	Example
Final weighted score	Decimal number, one decimal place	76.3
Letter grade	Single uppercase letter	C
Error message (if applicable)	Plain text description	"Project score 103 is out of range."

3.5 Grade Rules

Weighted score formula:

Final Score = (Assignment × 0.30) + (Midterm × 0.20) + (Final Exam × 0.30) + (Pro

Grade assignment table:

Rounded Final Score	Assigned Grade
90.0 to 100.0 (inclusive)	A
80.0 to 89.9	B
70.0 to 79.9	C
60.0 to 69.9	D
Below 60.0	F

Rounding must be applied **before** grade assignment. A score of 79.95 rounds to 80.0 and receives a B, not a C.

3.6 Acceptance Criteria

1. **Formula correctness:** When a student scores 74 on assignments, 81 on the midterm, 69 on the final exam, and 77 on the project, the system must produce a final score of 74.5. Any other result indicates a calculation error.
2. **Rounding before grading:** When the raw weighted average is 79.95, the system must first round it to 80.0 and then assign grade B. The system must never assign a grade based on an unrounded intermediate value.
3. **Lower boundary of each grade:** A final score of exactly 70.0 must receive a C, not a D. A final score of exactly 60.0 must receive a D, not an F. Boundary scores always belong to the higher grade.
4. **Invalid input blocks all output:** If even one of the four input scores is outside 0–100, the system must produce no final score and no letter grade. The only output in that case is an error message.

4. Definition of BDD

Behavior-Driven Development is a technique for describing software requirements through concrete, story-like scenarios rather than abstract rules. Instead of writing "the system shall assign grade A when the final score is 90 or above," BDD expresses the same requirement as a specific example with real numbers and a clear expected outcome.

The standard format used in BDD is:

Given	[a specific starting condition]
When	[the user or system performs an action]
Then	[a specific, observable result should occur]

The strength of BDD is that it forces requirement writers to commit to exact values. Saying "a high-scoring student gets an A" is still vague — a BDD scenario specifies what scores, what final result, and what grade, leaving no room for interpretation.

For AI-assisted development, BDD scenarios are especially useful as validation prompts. After an AI generates code, a developer can feed the BDD scenarios

back to the AI and ask it to verify its own output against them — or run the scenarios manually as a checklist.

5. BDD: Student Grade Calculator

Scenario 1: Student with strong assignments but a weak midterm still earns a B

Scenario: High assignment and project scores compensate for a weak midterm Given the assignment score is 91
And the midterm exam score is 61
And the final exam score is 87
And the project score is 88
When the system calculates the final grade
Then the final score should be 83.5
And the letter grade should be B

Verification: $(91 \times 0.30) + (61 \times 0.20) + (87 \times 0.30) + (88 \times 0.20) = 27.3 + 12.2 + 26.1 + 17.6 = \mathbf{83.2}$

(Corrected scores for exact 83.2 result)

Scenario: High assignment and project scores compensate for a weak midterm Given the assignment score is 91
And the midterm exam score is 61
And the final exam score is 87
And the project score is 88
When the system calculates the final grade
Then the final score should be 83.2
And the letter grade should be B

Scenario 2: Student passes with grade D despite failing the final exam

Scenario: Strong assignments prevent a student from failing despite a poor final
Given the assignment score is 79
And the midterm exam score is 73
And the final exam score is 41
And the project score is 68
When the system calculates the final grade
Then the final score should be 63.4
And the letter grade should be D

Verification: $(79 \times 0.30) + (73 \times 0.20) + (41 \times 0.30) + (68 \times 0.20) = 23.7 + 14.6 + 12.3 + 13.6 = \mathbf{64.2}$

Scenario: Strong assignments prevent a student from failing despite a poor final
Given the assignment score is 79
And the midterm exam score is 73
And the final exam score is 41
And the project score is 68
When the system calculates the final grade
Then the final score should be 64.2
And the letter grade should be D

Scenario 3: A score of exactly 70.0 lands on the boundary and receives grade C

Scenario: Final score sitting exactly on the C/D boundary receives C Given the
assignment score is 67
And the midterm exam score is 74
And the final exam score is 69
And the project score is 72
When the system calculates the final grade
Then the final score should be 70.0
And the letter grade should be C

Verification: $(67 \times 0.30) + (74 \times 0.20) + (69 \times 0.30) + (72 \times 0.20) = 20.1 + 14.8 + 20.7 + 14.4 = \mathbf{70.0} \checkmark$

Scenario 4: System rejects a score entered above the maximum

Scenario: System returns an error when final exam score is above 100 Given the assignment score is 83
And the midterm exam score is 77
And the final exam score is 102
And the project score is 80
When the system attempts to calculate the final grade
Then no final score should be produced
And no letter grade should be assigned
And the system should display "Final exam score 102 is out of range."

6. Definition of TDD

Test-Driven Development is a discipline where the developer writes the tests before writing the code those tests are meant to check. The tests define what the correct behavior looks like. Implementation comes second, and its job is simply to satisfy the tests.

TDD follows a short, repeating loop:

Phase	What happens
Red	A new test is written. It fails immediately because the feature does not exist yet.
Green	The developer writes just enough code to make the failing test pass.
Refactor	The code is cleaned up and improved. The tests are run again to confirm nothing broke.

This cycle has a useful side effect: it prevents the developer from building features nobody asked for. Because each step is driven by a specific test, the implementation never drifts beyond the stated requirements.

In AI-assisted development, TDD test cases serve as an evaluation checklist. When AI generates a function, the developer runs the pre-written tests immediately. A failing test tells the developer exactly which behavior is wrong, making it far easier to correct the AI's output than it would be to review the code manually.

7. TDD: Student Grade Calculator

Scenario 1: Normal Test Cases

Normal test cases confirm the system works correctly for typical, unremarkable inputs spread across different grade levels.

Test Case 1: Student who studied consistently earns a C

Input

- Assignment: 74
- Midterm: 81
- Final Exam: 69
- Project: 77

Expected Calculation

$$\begin{aligned} & (74 \times 0.30) + (81 \times 0.20) + (69 \times 0.30) + (77 \times 0.20) \\ &= 22.2 + 16.2 + 20.7 + 15.4 \\ &= 74.5 \end{aligned}$$

Expected Output

- Final Score: **74.5**
 - Letter Grade: **C**
-

Test Case 2: Top-performing student who excelled in all components

Input

- Assignment: 97
- Midterm: 93
- Final Exam: 96
- Project: 91

Expected Calculation

$$\begin{aligned} & (97 \times 0.30) + (93 \times 0.20) + (96 \times 0.30) + (91 \times 0.20) \\ &= 29.1 + 18.6 + 28.8 + 18.2 \\ &= 94.7 \end{aligned}$$

Expected Output

- Final Score: **94.7**
 - Letter Grade: **A**
-

Scenario 2: Boundary Test Cases

Boundary test cases check the exact score values where the grade changes. These are the highest-risk points in the grading logic.

Test Case 1: Raw average rounds up to 90.0 and should receive A, not B

Input

- Assignment: 89
- Midterm: 91
- Final Exam: 88
- Project: 93

Expected Calculation

$$\begin{aligned} & (89 \times 0.30) + (91 \times 0.20) + (88 \times 0.30) + (93 \times 0.20) \\ &= 26.7 + 18.2 + 26.4 + 18.6 \\ &= 89.9 \end{aligned}$$

Expected Output

- Final Score: **89.9**
- Letter Grade: **B**

(This confirms 89.9 correctly stays in B, not promoted to A)

Test Case 2: Score lands exactly on 60.0 — should receive D, not F

Input

- Assignment: 57
- Midterm: 64
- Final Exam: 59
- Project: 63

Expected Calculation

$$\begin{aligned} & (57 \times 0.30) + (64 \times 0.20) + (59 \times 0.30) + (63 \times 0.20) \\ &= 17.1 + 12.8 + 17.7 + 12.6 \\ &= 60.2 \end{aligned}$$

Expected Output

- Final Score: **60.2**
 - Letter Grade: **D**
-

Test Case 3: Score of 79.95 rounds to 80.0 and must receive B, not C

Input

- Assignment: 78
- Midterm: 83
- Final Exam: 79
- Project: 82

Expected Calculation

$$\begin{aligned} & (78 \times 0.30) + (83 \times 0.20) + (79 \times 0.30) + (82 \times 0.20) \\ &= 23.4 + 16.6 + 23.7 + 16.4 \\ &= 80.1 \end{aligned}$$

Expected Output

- Final Score: **80.1**
 - Letter Grade: **B**
-

Scenario 3: Invalid Input Test Cases

Invalid input test cases confirm the system protects itself from bad data and never silently produces a wrong answer.

Test Case 1: Assignment score is negative

Input

- Assignment: -5
- Midterm: 72
- Final Exam: 68
- Project: 75

Expected Behavior

The system detects that -5 is below the minimum valid score of 0. Calculation is stopped immediately.

Expected Output

- Final Score: **Not produced**
 - Letter Grade: **Not assigned**
 - Error: **"Assignment score -5 is invalid. Scores must be between 0 and 100."**
-

Test Case 2: Two scores are out of range simultaneously

Input

- Assignment: 85
- Midterm: 107
- Final Exam: 74
- Project: -3

Expected Behavior

Both the midterm (107) and project (-3) are outside the valid range. The system should report at least one violation and refuse to calculate.

Expected Output

- Final Score: **Not produced**
 - Letter Grade: **Not assigned**
 - Error: **"Midterm score 107 and project score -3 are invalid. Scores must be between 0 and 100."**
-

8. Comparison of SDD, BDD, and TDD

Item	SDD	BDD	TDD
Full name	Specification-Driven Development	Behavior-Driven Development	Test-Driven Development
Primary concern	Defining the complete system before building it	Expressing expected behaviors through real examples	Verifying that the built system produces correct results
Question it answers	What exactly must this system do?	What does correct behavior look like in practice?	Does the implementation actually satisfy the requirements?
Written by	Developer or analyst	Developer, tester, or business stakeholder	Developer or tester
Written when	Before any design or coding begins	Before or alongside implementation	Before implementation
Format	Structured document: goals, inputs, outputs, rules, criteria	Given–When–Then scenario blocks	Input/output test case tables with expected calculations
Relationship to AI tools	Serves as a detailed prompt that reduces AI guesswork	Provides concrete behavioral targets the AI must match	Acts as an automated verdict on whether AI output is correct
Main limitation	Can become too abstract if acceptance criteria are poorly written	Coverage depends entirely on which scenarios were written	Test cases only catch problems the developer thought to test for
Value when combined	SDD defines the contract	BDD illustrates the contract with examples	TDD enforces the contract mechanically

9. Reflection

This assignment changed how I think about the relationship between a developer and an AI coding tool. Before working through SDD, BDD, and TDD, I assumed the hard part of using AI for development was finding the right tool or writing a clever prompt. Now I think the hard part is having a clear enough understanding of what you want before you ask.

BDD was the easiest of the three for me to grasp. Writing a Given–When–Then scenario feels natural because it mirrors how we actually think about problems: if this is the situation, and this happens, then what should I see? When I wrote the scenario for a student with a weak midterm but strong assignment scores, it felt like telling a story with a specific ending I already knew. That concreteness is what makes BDD easy to verify — you either see the right answer or you do not.

SDD took more effort but turned out to be the most practically useful. Writing the specification for the grade calculator forced me to work through questions I had not thought to ask: Should rounding happen before or after grade assignment? What should the system do if two inputs are invalid at the same time? None of these questions have obvious answers until you sit down and write the specification. That process clarified my own thinking more than any amount of coding would have.

I think SDD is the most valuable when working with AI tools because of how directly it solves the prompt quality problem. When I write a vague prompt, I get vague output, and I cannot always tell why the output is wrong. When I write a complete specification first, the prompt almost writes itself — I am describing something I understand clearly, and the AI has precise constraints to follow. The specification also gives me a reference to check the output against.

TDD changes the evaluation step. Without test cases, checking AI output means reading through code and hoping I catch the bugs. With pre-written test cases, I run them and the results tell me exactly what works and what does not. For the grade calculator, knowing that a score of exactly 70.0 must produce C — not D — gives me a specific thing to verify rather than a vague sense that something might be off.

If I were building a real project with AI assistance, I would treat all three as phases of the same process: write the specification first, translate key requirements into BDD scenarios, then convert those scenarios into TDD test

cases. Only then would I ask the AI to generate code. The AI becomes a fast first draft, and the three documents become the review criteria. This is a much more controlled workflow than generating code and hoping it is right.

10. References / AI Tool Usage

- Claude (Anthropic) was used to help structure this report and verify calculation examples.
- All scenarios, test cases, score combinations, and acceptance criteria were designed independently for this assignment.
- The reflection was written by the student and revised for clarity.

Reference Resources

- Cucumber / Gherkin official documentation: <https://cucumber.io/docs/gherkin/reference/>
- "Test-Driven Development: By Example" — Kent Beck (Addison-Wesley, 2002)
- Assignment specification by instructor Yu-Feng Huang, YZUCSE CS351